特許出願説明資料 - 需要予測

クラスタリング × 時系列分解による需要予測精度・運用効率向上技術

1. 技術背景と課題

- 系列数が **販売チャネル × 商品カテゴリ × 地域/サービス種別** の積で爆発し、モデル管理が困難
- 個々の系列は観測点が少なくスパースであり、単独学習では精度が低下
- バッチ学習／推論時間が系列数に比例して増大し、クラウド課金が膨張

2. 本発明の構成要素

2-1. 時系列クラスタリング

項目	実装
距離尺度	DTW (Dynamic Time Warping)
手法	TimeSeriesKMeans
特徴量	月次需要総和行列 F (*)
クラスタ数決定	Silhouette Score 最大化

(*) 月次需要総和行列 F

💡 $$F \in \mathbb{R}^{N_{\text{series}} \times T}$$

- F

時系列クラスタリングで用いる特徴行列（feature matrix）。各行が 1 本の元系列を、各列が同じ時間位置を表す。内容は月次需要総和を Min-Max スケーリングした数値。

- $\mathbb{R}^{N_{\text{series}} \times T}$

行列 F が実数値（$\mathbb{R}$）で構成され、行数が N_{series}, 列数が T であることを示す集合記法。

- N_{series}

クラスタリング前に存在する元系列の本数。具体的には

販売チャネル × 商品カテゴリ × 店舗

の組合せ数と一致する。

- T

各系列の長さ（時点数）。例では「年月日」の軸で作ったため 1 年なら 365，4 年なら 1460 などになる。

簡潔に書けば

$F = [\text{系列ごとの月次需要}] \in \mathbb{R}^{(\text{系列本数}) \times (\text{時点数})}$

2-2. 多段 STL 時系列分解

1. 週次 (period=7) で STL 分解
2. 残差 + トレンドを入力として年次 (period=365) で再分解
3. 必要に応じて残差へ追加分解を再帰適用

生成特徴

`{seasonal_7, seasonal_365, trend, resid}`

Prophetの場合

timestamp	seasonal_7	seasonal_365	trend	resid	raw
2025-07-01	1	4	1	0.4	6.4
2025-07-02	2	6	2	0.1	
2026-07-01	8	2	18	0.3	???? → 28.3

seasonal_7を表現する関数を

f(t+1) = sin(theta_t - alpha) + beta

$f_{(t+1)} = \alpha * seasonal_t^7 + \mathrm{B} * X + ... + \epsilon$

forecast_(t+1) =

trend: (1,2,...,10)

diff(trend, 1): (undefined,1,...1)

2-3. 予測器

- Prophet (seasonality_mode="additive") or その他統計モデル
- 外生変数: 日本祝日フラグ、価格など
- 学習系列: 各クラスタの平均系列
- 予測範囲: 150 日先
- 評価指標: WAPE

2-X. クラスタごとの予測 → SKU単位の予測値への変換について

0. 各SKU（"すいか"など）ごとの分布（平均 μと標準偏差 σ）を保存
1. クラスタごとの予測値を標準化（平均0, 標準偏差1に変換）
   - 値域: [-1, 1]
   - $forecast^{normalized} = (forecast_{cluster} - \mu_{cluster})/\sigma_{cluster}$
2. 標準化されたクラスタごとの予測値を各SKUの予測値に変換
   - $forecast_{sku} = forecast_{cluster}^{normalized} * \sigma_{sku} + \mu_{sku}$

2-4. モデル選択の柔軟性

本プロセスは「クラスタリング ▶ 多段分解」で生成された特徴量を **モデル非依存** に提供できる。今回は Prophet を用いたが、以下のような任意の予測器へ容易に差し替え可能。

- 伝統的統計モデル：ARIMA / SARIMAX / ETS など
- 機械学習モデル：XGBoost / RandomForest / LightGBM など
- 深層学習モデル：LSTM / Temporal Fusion Transformer / N-BEATS など
  クラスタリングで系列数を圧縮しているため、計算コスト増大を伴う重いモデルでも実運用が可能になる。

3. システム構成とコード対応

```
project_root/
└─ src/forecaster/
   ├─ main.py        # 全体オーケストレーション
   ├─ cluster_data.py # Clustering クラス
   └─ forecast.py     # Forecaster クラス (STL + Prophet)
```

- GCP Cloud Functions でイベント駆動
- BigQuery から実績取得 → クラスタリング → 分解＋予測
- 結果／モデルを BigQuery・GCS に保存

> 大量でスパースな系列を「同じ絵柄ピースの束」にまとめ、重ねて透かし取る工程に例えられる。
>
> 1. **元のパズル**
> - ピース＝店舗×車種×サービスごとの需要系列
> - 盤面＝時間軸 (T ヶ月)
>
> 多くのピースは空白(0)が多く、全体像はぼやけている。
> 2. **クラスタリング＝仕分け**
>
> 似た模様のピースを自動で束ね、ピース総数 N_series をクラスタ数 N_c に圧縮。テーブルがスッキリする。
> 3. **多段 STL 分解＝透かし合わせ**
>
> 束を半透明紙に重ね、週次・年次など複数周期の波を順に写し取る。個別では見えなかった模様が浮き彫りになる。
> 4. **予測モデル＝色塗り**
>
> 抽出したトレンド・季節性という下描きがあれば、Prophet でも ARIMA でも LSTM でも好きな"絵の具"で塗れる。
>
> 結果:
>
> - 整理前: 10,000 ピース × 48 ヶ月 → ノイズだらけ
> - 整理後: 10 束 × 48 ヶ月 + 抽出周期成分 → データ量 1/1000、パターン鮮明
>
> 散らかったジグソーパズルを束ねて透かし合わせ、下絵をくっきりさせてから描き込む——これが本技術の圧縮・整列プロセスである。

4. 技術的効果

モデル数

- 従来: N_{series}
- 提案手法: $N_c \ll N_{\text{series}}$（ここで、$N_c$はクラスタ数）

学習／推論時間

- 従来: 線形に系列数へ依存
- 提案手法: 線形にクラスタ数へ依存

WAPE（予測精度）

- 従来: 基準値
- 提案手法: 7〜20% 改善 (社内検証)

傾向・周期性抽出

- 従来: スパースゆえノイズ多く不安定
- 提案手法: 圧縮後はパターンが鮮明化し、多段 STL が高精度に分解

5. 実施形態例

1. **サーバレス構成**: Cloud Scheduler / EventBridge → Function as a Service → DWH/GCS/S3 等
2. **オンプレ構成**: Airflow DAG + Spark/Hadoop クラスタでクラスタリングを分散実行
3. **リアルタイム構成**: Kafka / PubSub ストリームに載せた時系列を随時クラスタリング・予測し、API で提供

---

6. クレーム（ドラフト）

1. **請求項１**
   DTW 距離を用いる時系列クラスタリング手段と、多段 STL 分解により抽出した特徴量を用いて需要を予測する予測手段を備える需要予測装置。
   a. 補足: 今回はDTW距離を用いたK-meansクラスタリングにより上記を実現したが、時系列データをクラスタリングできる手法であれば、いずれでもかまわない。
      例: K-メドイド, ゼロショットLLMクラスタリング, k近傍etc
2. **請求項２**
   ~~クラスタ数をシルエット係数最大化で決定する請求項１記載の装置。~~
3. **請求項３**
   ~~予測手段が Prophet モデルを採用し、祝日情報を外生変数として用いる請求項１または２記載の装置。~~
   ~~どのような予測器にも（xxxという構成で, how）適用可能な、時系列データの特徴抽出装置~~

---

7. まとめ

- **大規模スパースデータをクラスタリングで圧縮→トレンド・季節性が浮き彫り→多段分解の効率と精度が向上** という一連の流れが発明のコア
  +* **系列圧縮 (クラスタリング)** と **多段季節分解 (STL)** の組み合わせにより、モデル運用負荷と計算コストを劇的に削減しつつ予測精度を向上
- 圧縮のおかげで高負荷モデルを含む **あらゆる予測アルゴリズムを選択可能** という柔軟性を持つ
- クラウド・オンプレ双方で実装可能な汎用性を有する

---

8. 先行技術と比較

以下は公開情報から確認できた関連技術の抜粋であり、本発明との差異を示す。

分類	文献・URL	技術概要	本発明との差異
ライブラリ実装	tslearn.clustering.silhouette_score https://tslearn.readthedocs.io	DTW/EUCLIDEAN/Soft-DTW 距離を用いたクラスタリング評価 API を提供	Silhouette は本発明でも採用だが、多段 STL 分解やクラスタ数自動決定ロジックは未実装
チュートリアル記事	"Time Series Clustering with tslearn" Medium, 2025-01-30	TimeSeriesKMeans と Silhouette によるクラスタリング手順紹介	実装例のみで、モデル数削減/分解による精度向上やシステム運用については未言及
学術論文	"A Benchmark Study on Time Series Clustering" arXiv:2004.09546 (2020)	112 UCR データセットで 8 手法を比較し、距離・アルゴリズムの選択影響を定量評価	ベンチマークに留まり、クラスタリング後の予測・モデル管理には踏み込まない
応用研究	"DTW k-means clustering for fault detection in photovoltaic modules" arXiv:2306.08003 (2023)	DTW-KMeans で太陽光モジュール故障時系列を	需要予測用途ではなく、多段分解も無い

分類	文献・URL	技術概要	本発明との差異
		クラスタリングし異常検知	

上記から分かるように、DTW×KMeans や Silhouette によるクラスタ数評価は既知であるが、

1. クラスタリングによる **モデル本数削減** と **学習・推論時間の線形化** を大規模運用に適用
2. **STL を複数周期で再帰的に適用** し、クラスタ平均系列でも季節性を分離して精度を向上
3. **サーバレス実装** によりクラウドコストと運用負荷を抑制
   を同時に実現する事例は見当たらず、本発明の新規性・進歩性を裏付ける。

---

9. 参考文献

1. tslearn Documentation, "silhouette_score", https://tslearn.readthedocs.io (accessed 2025-10-XX)
2. İ. Şahbazoğlu, "Time Series Clustering with tslearn", Medium, 2025-01-30.
3. A. Javed et al., "A Benchmark Study on Time Series Clustering", arXiv:2004.09546v2, 2023.
4. M. Author et al., "DTW k-means clustering for fault detection in photovoltaic modules", arXiv:2306.08003, 2023.

10. 先行技術（詳細リスト）

10-1. 関連特許

公開番号	出願人 / 年	技術ポイント	本技術との差異
US2020/0294067 (US11727420B2)	Target Brands Inc., 2019	DTWベース時系列クラスタリングで販売データをクラスタ化し新商品の需要を予測	予測対象を「短系列のみ」の場合に限る点と、STL等の分解を含まず。クラスタ数はCVIsで最適化だが季節要因抽出が弱い
US10521809B2	Walmart Apollo, 2015	小売SKUの時系列を自動グルーピングし階層的予測を実施	グループ化アルゴリズムは距離学習だが、モデル圧縮効果やマルチ周期分解は未提案
US2020/0074485 (Oracle)	Oracle, 2018	需要予測パラメータのメタ最適化（ハイパーパラメータ探索）	予測モデル個別最適化に注力し、系列圧縮やSTL多段分解は提案なし
US11200607B2	Walmart Apollo, 2019	多モデル異常検知と需要予測を組み合わせて小売異常を検出	DTW クラスタリング無し、分解技術無し
JP6810745B2	Blue Yonder, 2021	異種属性を持つ顧客時系列のターゲットクラスタリング	業務対象が顧客セグメントで、時間分解やモデル数削減に踏み込まず

10-2. 学術論文

文献	年	概要	本技術との差異
Paparrizos & Gravano "K-Shape" SIGMOD 2015	2015	位相不変シェイプ距離で時系列クラスタリング	需要予測への応用・分解処理なし
Begum et al. "TADPole: Density based time series clustering" 2016	2016	DTW距離の剪定によりDensity Peaksを高速化	クラスタ数自動選択・予測フェーズ未実装
Javed et al. "Benchmark Study on Time Series Clustering" arXiv 2023	2023	112データセットで8手法を比較	分解や予測統合せず、圧縮効果評価無し
Hyndman et al. "Forecasting hierarchical time series" IJF 2011	2011	集約(Top-down/Bottom-up)で階層予測	事前クラスタリング無し、STL多段分解無し
Cleveland et al. "STL: Seasonal-Trend Decomposition" JASA 1990	1990	単一周期STL分解アルゴリズムを提案	我々は複数周期×再帰的適用に拡張
Taylor & Letham "Prophet" 2018	2018	休日・トレンドを考慮する加法/乗法モデル	当発明ではProphetを代表例として採用、モデル自由度を強調

10-3. 位置付け

1. 既存特許はいずれも「クラスタリング」か「予測パラメータ最適化」の片側に寄り、本発明のように
   ①DTWクラスタリングで系列数を劇的に削減しつつ
   ②多周期・再帰STLでトレンド/季節性を強調し

③モデル種別を問わず投入できる
という統合フローは見当たらない。

2. 学術的にもクラスタリングと分解を連携させた需要予測フレームワークの報告は稀であり、業務実装(サーバレスGCF)を含む点で新規性・進歩性が高い。